Conferencia 9: recursión

Programación · Ciencia de la Computación y Ciencia de Datos · MatCom · 2026-11-11

Hay tres varillas clavadas en una tabla. En la primera hay ocho discos de tamaños distintos, ensartados del más grande abajo al más pequeño arriba. Hay que pasarlos todos a la tercera varilla con dos reglas: se mueve un disco cada vez, el de arriba de alguna varilla, y nunca se pone un disco sobre otro más pequeño. Es el juego de las **Torres de Hanói**.

Con un disco es un movimiento. Con dos, tres: el pequeño a la del medio, el grande a la tercera, el pequeño encima. Con tres discos ya hay que pensar un rato, y con ocho casi nadie lo hace a mano sin equivocarse. Intenta escribir un programa con `while` y `for` que imprima los movimientos para `n` discos, y verás que no es nada obvio qué repetir.

Al final de la clase lo resolvemos en cinco líneas. Para llegar ahí hace falta una sola idea nueva: **una función que se llama a sí misma**. La clase tiene dos mitades. En la primera vemos funciones que se llaman a sí mismas una vez, y descubrimos que esas son ciclos escritos de otra manera. En la segunda vemos funciones que se llaman a sí mismas más de una vez, que es donde la recursión resuelve lo que los ciclos no resuelven fácil.

1. Una función que se llama a sí misma

En los ejercicios de la conferencia 3 escribiste el máximo común divisor. El algoritmo de Euclides dice que el `mcd` de `a` y `b` es el mismo que el de `b` y el resto de dividir `a` entre `b`, y que el `mcd` de cualquier número con 0 es el propio número. Se puede escribir tal cual:

```python
def mcd(a, b):
    """Máximo común divisor de a y b, por el algoritmo de Euclides."""
    if b == 0:
        return a
    return mcd(b, a % b)

print(mcd(84, 36))
print(mcd(17, 5))
```

```
12
1
```

Dentro de `mcd` se llama a `mcd`. Es correcto, y no da vueltas para siempre, por dos razones que conviene separar:

- Hay un **caso base**, `b == 0`, que se responde sin llamar a nadie.
- El **caso recursivo** llama a la función con un problema más pequeño: el segundo argumento pasa de `b` a `a % b`, que es menor que `b`. Como los números naturales no pueden achicarse para siempre, en algún momento el segundo argumento llega a 0.

Son las mismas dos piezas que hacen terminar a un `while`, la condición de salida y algo que se achica en cada vuelta, con otros nombres. Veamos qué pasa con `mcd(84, 36)`:

```python
def mcd_traza(a, b):
    print(f"mcd({a}, {b})")
    if b == 0:
        return a
    return mcd_traza(b, a % b)
```

```
print(mcd_traza(84, 36))
```

```
mcd(84, 36)
mcd(36, 12)
mcd(12, 0)
12
```

Tres llamadas, cada una con números más pequeños, hasta que la última encuentra el caso base y devuelve 12. Ese 12 es la respuesta de la penúltima llamada, que lo devuelve sin tocarlo, y de la anterior, y así hasta la primera.

2. La búsqueda binaria, otra vez

En la conferencia 2 la computadora adivinaba un número partiendo el rango a la mitad, y en la conferencia 8 escribimos la búsqueda binaria con un `while` y dos índices. Se puede pensar de otra forma: buscar `x` entre `izq` y `der` es mirar el medio y, si no es, **buscar `x` en una de las dos mitades**. Buscar en una mitad es el mismo problema, más pequeño.

```
def buscar_binaria(lista, x, izq, der):
    """Busca x en lista[izq..der], que está ordenada. Devuelve su índice, o -1."""
    if izq > der:
        return -1                       # el pedazo está vacío
    medio = (izq + der) // 2
    if lista[medio] == x:
        return medio
    elif lista[medio] < x:
        return buscar_binaria(lista, x, medio + 1, der)
    else:
        return buscar_binaria(lista, x, izq, medio - 1)

primos = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
print(buscar_binaria(primos, 13, 0, len(primos) - 1))
print(buscar_binaria(primos, 12, 0, len(primos) - 1))
```

```
5
-1
```

Compárala con la de la conferencia 8. Lo que allá era `izq = medio + 1` aquí es una llamada con `medio + 1` en el lugar de `izq`. La condición del `while`, `izq ⩽ der`, aquí es su contraria, `izq > der`, en el caso base. Es el mismo algoritmo.

Tener que escribir `0, len(primos) - 1` en cada llamada es incómodo. Lo normal es darles a los dos índices un valor por defecto, como en la conferencia 3. Para `izq` es fácil, `izq=0`. Para `der` no, porque el valor que queremos, `len(lista) - 1`, depende de otro parámetro, y un valor por defecto no puede depender de los otros parámetros. El truco es darle a `der` el valor `None` y calcular el de verdad dentro de la función:

```
def buscar_binaria(lista, x, izq=0, der=None):
    """Busca x en lista[izq..der], que está ordenada. Devuelve su índice, o -1."""
    if der is None:
        der = len(lista) - 1            # no nos dijeron dónde termina: hasta el final
    if izq > der:
        return -1
    medio = (izq + der) // 2
    if lista[medio] == x:
        return medio
```

```
    elif lista[medio] < x:
        return buscar_binaria(lista, x, medio + 1, der)
    else:
        return buscar_binaria(lista, x, izq, medio - 1)

print(buscar_binaria(primos, 29))
```

```
9
```

`None` es un valor de Python que significa "no hay valor". Es lo que devuelve una función que no tiene `return`, como viste en la conferencia 3. Para preguntar si una variable vale `None` se escribe `der is None`, no `der == None`; las dos funcionan aquí, pero `is` es la forma que vas a ver en todo el código de Python. Solo la primera llamada recibe `der = None`. Las recursivas ya le pasan un número.

3. La pila de llamadas

El factorial de `n` es $n! = 1 \cdot 2 \cdot 3 \cdots n$, y $0! = 1$. También se puede definir así: $0! = 1$, y $n! = n \cdot (n-1)!$. Esa segunda definición se escribe en Python sin cambiar nada:

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)

print(factorial(4))
print(factorial(20))
```

```
24
2432902008176640000
```

Hay una diferencia con `mcd` que no se ve a primera vista. En `mcd`, la llamada recursiva era lo último que pasaba: su resultado se devolvía tal cual. En `factorial`, cuando `factorial(4)` llama a `factorial(3)`, todavía le falta algo por hacer: multiplicar lo que le devuelvan por 4. Así que `factorial(4)` no puede terminar. Se queda **esperando**, con su `n = 4` guardado, mientras `factorial(3)` trabaja. Y `factorial(3)` se queda esperando a `factorial(2)`, y así hasta `factorial(0)`, que devuelve 1 sin esperar a nadie.

En ese momento hay cinco llamadas a `factorial` vivas a la vez, cada una con su propia `n`. Python las guarda en orden, cada una a continuación de la que la llamó, en lo que se llama la **pila de llamadas**. En la figura de abajo cada llamada nueva baja un escalón. Cuando `factorial(0)` devuelve 1, sale de la pila, y la llamada que la esperaba, un escalón más arriba, multiplica `1 * 1` y devuelve 1; la siguiente hacia arriba multiplica `2 * 1`, y así hasta la primera.

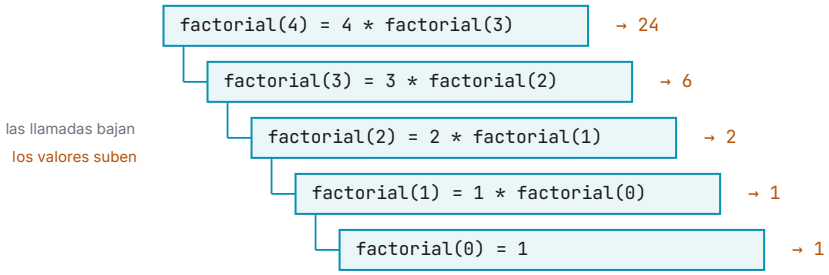


Figura 1. La pila de `factorial(4)`. Cada llamada baja un escalón y se queda esperando. A la derecha, lo que devuelve cada una cuando la de debajo le contesta: el valor cambia en cada escalón, porque cada llamada todavía tenía una multiplicación pendiente.

Lo importante de la figura 1 es que **cada llamada tiene sus propias variables**. La `n` de `factorial(4)` vale 4 y la de `factorial(2)` vale 2, a la vez, y no se pisan. Es lo mismo que viste en la conferencia 3 sobre las variables locales: cada vez que se llama una función, sus variables se crean de nuevo. Que la función que llama sea la misma no cambia nada.

4. Recursión de cola

Ahora escribe el factorial de otra manera. En lugar de dejar la multiplicación pendiente, la hacemos antes de llamar, y le pasamos el resultado parcial a la llamada siguiente en un parámetro nuevo, `acumulado`:

```
def factorial_cola(n, acumulado=1):
    if n == 0:
        return acumulado
    return factorial_cola(n - 1, acumulado * n)

print(factorial_cola(4))
print(factorial_cola(20))
```

```
24
2432902008176640000
```

El parámetro `acumulado` es el acumulador de la conferencia 2, el que llevaba la suma o el producto parcial en un ciclo. Ahora la llamada recursiva es otra vez lo último que hace la función, como en `mcd`, y el valor que devuelve el caso base sube por la pila sin que nadie lo toque:

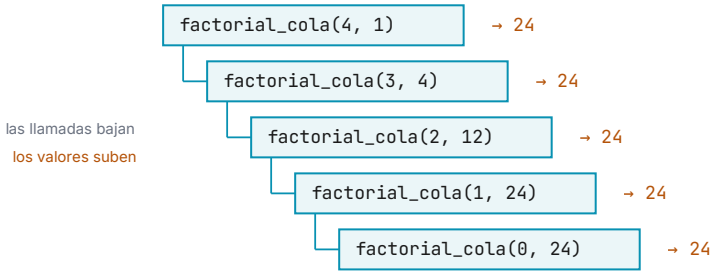


Figura 2. La pila de `factorial_cola(4)`. El trabajo se hace al bajar, en el segundo argumento. Al subir, todas las llamadas devuelven el mismo 24: ninguna tenía nada pendiente.

Una función donde la llamada recursiva es lo último que se hace, y su resultado se devuelve sin tocarlo, se llama **recursiva de cola**. `mcd` lo es, la búsqueda binaria recursiva lo es, y `factorial_cola` lo es. `factorial` no lo es, por la multiplicación pendiente.

Compara las dos figuras. En la figura 2, las llamadas de arriba no aportan nada cuando les llega la respuesta, así que no hacía falta guardarlas. Una función recursiva de cola se puede convertir en un ciclo de forma mecánica: los parámetros pasan a ser variables, el caso base es la condición de salida, y la llamada recursiva es una asignación de todos los parámetros a la vez.

```
def factorial_ciclo(n):
    acumulado = 1
    while n != 0:
        n, acumulado = n - 1, acumulado * n
    return acumulado

def mcd_ciclo(a, b):
    while b != 0:
        a, b = b, a % b
    return a

print(factorial_ciclo(20), mcd_ciclo(84, 36))
```

```
2432902008176640000 12
```

Pon `factorial_cola` y `factorial_ciclo` una al lado de la otra. La línea `return factorial_cola(n - 1, acumulado * n)` es `n, acumulado = n - 1, acumulado * n` con otra ropa. La asignación simultánea de la conferencia 4 importa aquí: si escribieras primero `n = n - 1` y después `acumulado = acumulado * n`, multiplicarías por el `n` nuevo, que es otro número.

Esa es la conclusión de la primera mitad: **una recursión de cola es un ciclo escrito de otra manera**. A veces se lee mejor, como en `mcd`, donde la versión recursiva es casi la definición matemática. Pero no hace nada que el ciclo no haga.

5. El límite de la pila

Y en Python, además, la versión recursiva tiene un problema que el ciclo no tiene:

```
print(len(str(factorial_ciclo(1500))), "cifras")

try:
    factorial_cola(1500)
except RecursionError as error:
    print(type(error).__name__, error)
```

```
4115 cifras
RecursionError maximum recursion depth exceeded
```

El `try` y el `except` son los de la conferencia 5, con una cosa nueva: `as error` le pone nombre al error que se atrapó, para poder mirarlo. `type(error).__name__` es el nombre de su tipo, y al imprimir `error` sale el mensaje que lo acompaña. Así vemos el error sin que el programa se muera.

El ciclo calcula $1500!$, un número de más de cuatro mil cifras, sin problema. La versión recursiva falla con `RecursionError`, aunque sea de cola. La pila de llamadas tiene un tope, y Python lo pone por defecto en mil llamadas:

```
print(sys.getrecursionlimit())
```

```
1000
```

El tope está para proteger a la máquina de una recursión que nunca llega al caso base, el equivalente del ciclo infinito. Fíjate en lo que pasa si olvidas el caso base, o si el problema no se achica:

```python
def factorial_roto(n):
    return n * factorial_roto(n - 1)    # sin caso base

try:
    factorial_roto(4)
except RecursionError as error:
    print(type(error).__name__, error)
```

```
RecursionError maximum recursion depth exceeded
```

Un ciclo infinito se queda colgado hasta que aprietas Ctrl+C. Una recursión infinita se para sola al llegar a mil llamadas, con un error que dice qué pasó.

Algunos lenguajes, como Scheme, reconocen las llamadas de cola y las convierten en ciclos por su cuenta, sin gastar pila. Python decidió no hacerlo, para que el error, cuando ocurre, muestre todas las llamadas que lo causaron. Así que en Python la recursión de cola tiene las desventajas de la recursión y ninguna ventaja sobre el ciclo, salvo a veces la claridad.

Con eso queda abierta la pregunta: si la recursión que hemos visto es un ciclo disfrazado, ¿para qué sirve? La segunda mitad la contesta.

6. Más de una llamada: las Torres de Hanói

Volvemos a los ocho discos. La idea que lo resuelve es pensar en el disco más grande. Para moverlo de la primera varilla a la tercera, los otros siete tienen que estar fuera de su camino, todos en la del medio. Así que el plan es:

1. Mover los siete discos de arriba de la primera varilla a la del medio.
2. Mover el disco grande de la primera a la tercera.
3. Mover los siete discos de la del medio a la tercera.

Los pasos 1 y 3 son el mismo problema con un disco menos y otras varillas. El disco grande no estorba, porque está abajo de todo y cualquier disco puede ponerse sobre él. Con cero discos no hay nada que hacer.

```python
def hanoi(n, origen, destino, auxiliar):
    """Imprime los movimientos para pasar n discos de origen a destino."""
    if n == 0:
        return
    hanoi(n - 1, origen, auxiliar, destino)
    print(f"disco {n}: {origen} -> {destino}")
    hanoi(n - 1, auxiliar, destino, origen)

hanoi(3, "A", "C", "B")
```

```
disco 1: A → C
disco 2: A → B
disco 1: C → B
```

```
disco 3: A → C
disco 1: B → A
disco 2: B → C
disco 1: A → C
```

Son cinco líneas, y resuelven el problema para cualquier cantidad de discos. Sigue los movimientos con tres monedas de tamaños distintos y comprueba que ninguno pone una moneda grande sobre una pequeña.

Lo que hace esta función distinta de todo lo anterior es que **se llama a sí misma dos veces**. La primera llamada no es la última cosa que se hace: después de ella viene un `print` y otra llamada. No es de cola, y no se puede convertir en un ciclo cambiando parámetros, porque después de resolver los `n - 1` discos de arriba hay que acordarse de que todavía falta mover el grande y los otros `n - 1`. Esa memoria es justo lo que guarda la pila de llamadas.

¿Cuántos movimientos hace? Contamos en lugar de imprimir:

```python
def movimientos(n):
    if n == 0:
        return 0
    return movimientos(n - 1) + 1 + movimientos(n - 1)

for n in range(1, 11):
    print(n, movimientos(n), 2 ** n - 1)
```

```
1 1 1
2 3 3
3 7 7
4 15 15
5 31 31
6 63 63
7 127 127
8 255 255
9 511 511
10 1023 1023
```

La tercera columna es $2^n - 1$, y coincide. Cada disco más duplica los movimientos y suma uno. Con ocho discos son 255. Hay una leyenda de un templo donde unos monjes mueven 64 discos de oro, un disco por segundo, y el mundo se acaba cuando terminen:

```python
segundos = 2 ** 64 - 1
print(f"{segundos / (3600 * 24 * 365.25):.3e} años")
```

```
5.845e+11 años
```

Más de quinientos mil millones de años, unas cuarenta veces la edad del universo.

El árbol de llamadas

En el factorial las llamadas formaban una fila, cada una esperando a la siguiente. En Hanói cada llamada hace dos, y las llamadas forman un **árbol**:

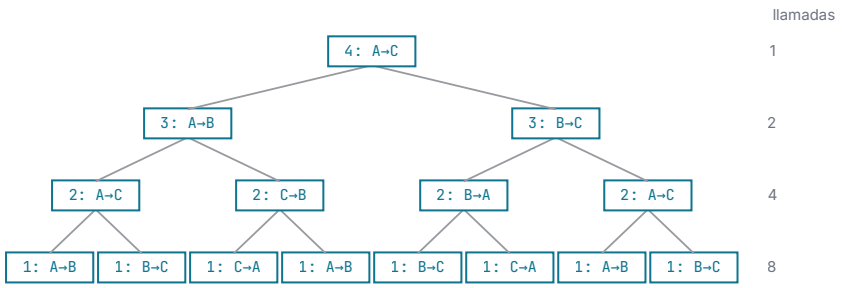


Figura 3. Las llamadas de `hanoi(4, "A", "C", "B")` que mueven algo. Cada nodo dice cuántos discos mueve y entre qué varillas. Cada llamada hace un movimiento propio, el del disco grande, y le encarga el resto a sus dos hijas. A la derecha, cuántas llamadas hay en cada nivel.

La figura 3 da otra manera de contar los movimientos. Cada llamada hace un solo movimiento propio, así que hay tantos movimientos como llamadas. En el primer nivel hay 1 llamada, en el segundo 2, en el tercero 4, en el cuarto 8: cada nivel duplica al anterior, y en total son $1 + 2 + 4 + 8 = 15 = 2^4 - 1$. Contar las llamadas nivel por nivel es la herramienta con la que la conferencia 10 va a explicar por qué la ordenación por mezcla es tan rápida.

7. Recursión sobre listas y cadenas

Antes del último problema volvemos un momento a funciones con una sola llamada, para que la recursión no quede asociada solo a los números. La recursión no es solo para números. Una lista que no está vacía es su primer elemento seguido de una lista más corta, y eso basta para recorrerla:

```
def suma(lista):
    if len(lista) == 0:
        return 0
    return lista[0] + suma(lista[1:])

print(suma([4, 2, 5, 3]))
```

```
14
```

La rebanada `lista[1:]` es la lista sin su primer elemento, y es el problema más pequeño. Esta suma tiene el mismo defecto que el factorial: no es de cola, y con más de mil elementos se rompe. Y además cada rebanada copia la lista, como viste en la conferencia 4. Para sumar una lista, el ciclo es mejor.

Lo mismo vale para cadenas. Una palabra es un palíndromo si la primera letra es igual a la última y lo que queda en medio es un palíndromo:

```
def es_palindromo(palabra):
    if len(palabra) <= 1:
        return True
    return palabra[0] == palabra[-1] and es_palindromo(palabra[1:-1])

for palabra in ["reconocer", "anilina", "radar", "recursion"]:
    print(palabra, es_palindromo(palabra))
```

```
reconocer True
anilina True
```

```
radar True
recursion False
```

El caso base es `len(palabra) ⩽ 1` y no `== 0`, porque cada llamada quita dos letras: si la palabra tiene una cantidad impar de letras, se llega a una sola letra, nunca a la cadena vacía.

Lo que un ciclo no hace fácil: aplanar

Ahora un problema donde la recursión sí gana. Una lista puede contener otras listas, que a su vez contienen otras, a cualquier profundidad:

```
anidada = [1, [2, 3], [4, [5, 6, [7]], 8], [[[9]]]]
```

Queremos una lista **plana** con los números en el mismo orden: `[1, 2, ..., 9]`. Con un ciclo recorres el primer nivel; con dos ciclos anidados, dos niveles. Pero no sabes cuántos niveles hay hasta que el programa corre, y no puedes escribir una cantidad de ciclos anidados que todavía no conoces. Es el mismo callejón de la conferencia 3, donde no se podía escribir una cantidad de variables que todavía no se conocía.

Con recursión es directo. Recorremos la lista, y cada elemento que sea a su vez una lista lo aplanamos con la misma función:

```
def aplanar(lista):
    plana = []
    for x in lista:
        if isinstance(x, list):
            plana.extend(aplanar(x))     # x es una lista: aplánala y agrégala
        else:
            plana.append(x)
    return plana

print(aplanar(anidada))
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

`isinstance(x, list)` pregunta si `x` es una lista, y `extend` agrega al final todos los elementos de otra lista, uno por uno. El caso base no está escrito con un `if` aparte: es una lista sin listas dentro, para la que el ciclo nunca entra a la rama recursiva.

Fíjate en que aquí la recursión está **dentro de un ciclo**: una llamada puede hacer cero, una o muchas llamadas, tantas como sublistas tenga. Cada llamada recuerda en la pila por dónde iba su ciclo, y eso es justo lo que con ciclos tendrías que llevar a mano.

8. Fibonacci y el reloj

La sucesión de Fibonacci empieza con 0 y 1, y cada término es la suma de los dos anteriores: 0, 1, 1, 2, 3, 5, 8, 13... La definición se escribe sola:

```
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print([fib(n) for n in range(12)])
```

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

Es correcta, y es una de las funciones más lentas que vas a escribir este año. Medimos:

```
tiempos = []
for n in range(20, 33, 2):
    inicio = time.perf_counter()
    fib(n)
    tiempos.append((n, time.perf_counter() - inicio))

for k, (n, t) in enumerate(tiempos):
    razon = f"×{t / tiempos[k - 1][1]:.1f}" if k > 0 else ""
    print(f"fib({n}): {t:7.3f} s  {razon}")
```

```
fib(20):   0.001 s
fib(22):   0.003 s  ×2.6
fib(24):   0.008 s  ×2.6
fib(26):   0.019 s  ×2.5
fib(28):   0.055 s  ×2.9
fib(30):   0.136 s  ×2.5
fib(32):   0.359 s  ×2.6
```

La expresión `a if condicion else b` vale `a` si la condición es verdadera y `b` si no. Es un `if` que se puede usar dentro de una expresión, y aquí sirve para no imprimir razón en la primera fila, que no tiene una anterior con qué compararse.

Cada dos términos más, el tiempo se multiplica por algo más de dos y medio. Con eso se puede estimar cuánto tardarían el término 40 y el 50:

```
n, t = tiempos[-1]
razon = (tiempos[-1][1] / tiempos[0][1]) ** (1 / (len(tiempos) - 1))
for objetivo in [40, 50]:
    estimado = t * razon ** ((objetivo - n) / 2)
    print(f"fib({objetivo}): unos {estimado:.0f} segundos, {estimado / 60:.1f} minutos")
```

```
fib(40): unos 22 segundos, 0.4 minutos
fib(50): unos 3026 segundos, 50.4 minutos
```

La segunda línea calcula la razón media. Del primer tiempo al último hubo seis saltos de dos términos, y en total el tiempo se multiplicó por `tiempos[-1][1] / tiempos[0][1]`. Si cada salto multiplica por la misma razón r, entonces r^6 es ese total, y r es su raíz sexta. En Python la raíz sexta se escribe como potencia: `** (1 / 6)`. Después, para llegar de 32 a 40 faltan cuatro saltos más, y se multiplica por r^4.

La razón está en el árbol de llamadas. `fib(5)` llama a `fib(4)` y a `fib(3)`, pero `fib(4)` también llama a `fib(3)`, que se calcula dos veces desde cero:

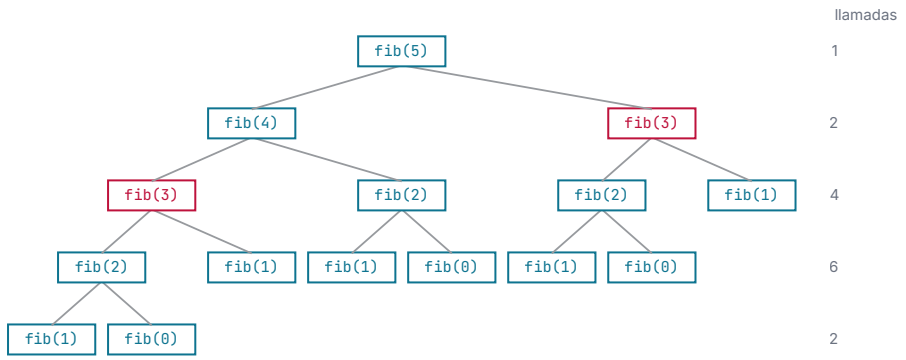


Figura 4. Las llamadas de `fib(5)`. En rojo, las dos veces que se calcula `fib(3)`, cada una con todo su subárbol. A la derecha, cuántas llamadas hay en cada nivel.

En la figura 4, `fib(3)` aparece dos veces, `fib(2)` tres veces y `fib(1)` cinco. Contemos cuántas llamadas hace `fib(n)` en total:

```python
def llamadas(n):
    if n < 2:
        return 1
    return 1 + llamadas(n - 1) + llamadas(n - 2)

for n in [10, 20, 30]:
    print(f"fib({n}) = {fib(n)}, con {llamadas(n)} llamadas")
```

```
fib(10) = 55, con 177 llamadas
fib(20) = 6765, con 21891 llamadas
fib(30) = 832040, con 2692537 llamadas
```

Para `fib(30)` son más de dos millones de llamadas para calcular un número que con un ciclo sale en treinta sumas. El problema no es la recursión: es que el árbol repite trabajo. Cada vez que se pide `fib(3)`, se calcula entero otra vez, aunque ya se haya calculado antes. Hanói también tiene un árbol que crece como 2^n, pero ahí no hay trabajo repetido: los $2^n - 1$ movimientos son todos necesarios. En Fibonacci casi todas las llamadas sobran.

Cómo arreglarlo, recordando cada respuesta en lugar de volverla a calcular, es el tema de programación dinámica, en el segundo semestre. Por ahora queda la observación: **una definición recursiva correcta puede ser un programa inútil**, y lo que lo decide es el árbol de llamadas.

9. Resumen

- Una función recursiva se llama a sí misma. Necesita un **caso base**, que se responde sin llamar a nadie, y un **caso recursivo** que llama con un problema más pequeño.
- Cada llamada tiene sus propias variables. Las llamadas que esperan se guardan en la **pila de llamadas**.
- Una función es **recursiva de cola** si la llamada recursiva es lo último que hace y su resultado se devuelve sin tocarlo. `mcd`, la búsqueda binaria y `factorial_cola` lo son; `factorial` no.
- Una recursión de cola se convierte en un ciclo de forma mecánica: los parámetros son las variables y la llamada es una asignación simultánea. Un acumulador convierte muchas recursiones en recursiones de cola.
- Python pone un tope de mil llamadas a la pila, y no elimina las llamadas de cola. Pasar el tope da `RecursionError`, igual que olvidar el caso base.

- Una función que se llama **más de una vez** hace un árbol de llamadas. Es lo que resuelve Hanói y lo que aplana una lista anidada, cosas que un ciclo no hace fácil.
- Contar las llamadas nivel por nivel en el árbol da el costo de una función recursiva: $2^n - 1$ movimientos en Hanói.
- Si el árbol repite subproblemas, como en Fibonacci, la recursión puede ser correcta y a la vez inútil.

Ejercicios

1. Escribe `potencia(a, n)` que calcule a^n con recursión, usando $a^0 = 1$ y $a^n = a \cdot a^{n-1}$. Después escribe la versión de cola con un acumulador, y después la versión con un ciclo que sale de convertir la de cola.
2. Escribe `contar_digitos(n)` y `suma_digitos(n)` recursivas, para un entero `n >= 0`. ¿Qué es el problema más pequeño? Pista: `n // 10` y `n % 10`.
3. Escribe `invertir(cadena)` recursiva, que devuelva la cadena al revés. Después escribe `maximo(lista)` recursiva, sin usar `max`. ¿Cuál es el caso base de `maximo`, si una lista vacía no tiene máximo?
4. La búsqueda binaria recursiva tiene tantas llamadas en la pila como vueltas daba la del `while`. ¿Con qué tamaño de lista daría `RecursionError`? Justifícalo sin ejecutarla.
5. Modifica `hanoi` para que no imprima los movimientos sino que devuelva una lista de tuplas `(disco, origen, destino)`. Después escribe `comprobar(n, movimientos)` que simule las tres varillas con tres listas y diga si la secuencia es legal y termina con todos los discos en la tercera. Úsala para comprobar tu `hanoi` hasta 10 discos.
6. Escribe `profundidad(lista)` que diga cuántos niveles de anidamiento tiene una lista: `[1, 2]` tiene profundidad 1, `[1, [2]]` tiene 2 y la lista `anidada` de la sección 7 tiene 4. ¿Qué profundidad tiene la lista vacía?
7. Escribe `fib_ciclo(n)` con un ciclo y dos variables, y compara su tiempo con el de `fib` para `n = 30`. ¿Cuántas sumas hace cada una?
8. Escribe una versión de `suma(lista)` que sea de cola, con un acumulador y un índice en lugar de rebanadas. Conviértela en un ciclo. ¿Cuál de las tres versiones puede sumar una lista de un millón de elementos?